

Playwright + MCP

The Complete Master Sheet for
AI-Powered Browser Automation

What's inside:

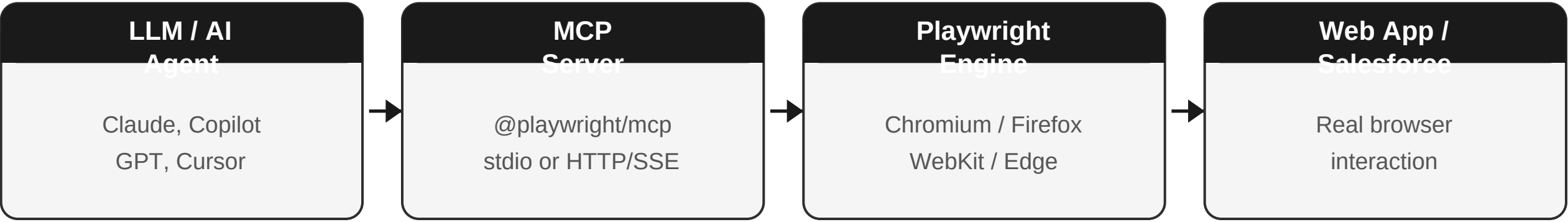
01	What is Playwright MCP? — Architecture
02	The 3-Step Loop + Two Server Types
03	Setup + Configuration Reference
04	Locators, Actions & MCP Tools
05	Advanced Concepts + CLI Quick Ref

Swipe to explore >>

5 slides • 2025

AI brain (LLM) + MCP Protocol + Playwright Engine = Intelligent Browser Automation

Architecture Flow



Two Types of MCP Server



3 Ways to Install

Option 1 — Claude Desktop / VS Code (Recommended)

```

{ "mcpServers": { "playwright": {
  "type": "stdio", "command": "npx",
  "args": ["-y", "@playwright/mcp@latest"] } } }
```

Option 2 — VS Code CLI (One-liner)

```

code --add-mcp '{"name":"playwright",
  "command":"npx","args":["@playwright/mcp@latest"]}'
```

Option 3 — ExecuteAutomation package

```

npm install -g @executeautomation/playwright-mcp-server
```

Key MCP Config Options

Option	Values	Default	Purpose
type	stdio / http	stdio	Transport mode
port	Any port number	8931	HTTP/SSE port
headless	true / false	true	Show browser window
browser	chromium/firefox/webkit	chromium	Browser engine
--device	iPhone 15 / Pixel 7	—	Device emulation
--viewport	1280x720 etc.	—	Custom viewport size
--save-trace	true / false	false	Save Playwright trace
--output-dir	folder path	—	Screenshot/trace folder

Built-in MCP Tools (Snapshot Mode)

Tool	What it does
<code>browser_navigate</code>	Go to a URL
<code>browser_click</code>	Click an element
<code>browser_type</code>	Type text into field
<code>browser_snapshot</code>	Capture accessibility tree
<code>browser_screenshot</code>	Take a screenshot
<code>browser_select</code>	Choose dropdown option
<code>browser_hover</code>	Hover over element
<code>browser_wait</code>	Wait for condition
<code>browser_scroll</code>	Scroll page/element
<code>browser_evaluate</code>	Run JavaScript in page

Playwright Actions (via MCP)

Action	Example
<code>goto()</code>	<code>page.goto('/login')</code>
<code>click()</code>	<code>page.click('#btn')</code>
<code>fill()</code>	<code>page.fill('#input','val')</code>
<code>press()</code>	<code>page.press('#f','Enter')</code>
<code>selectOption()</code>	<code>page.selectOption()</code>
<code>check()</code>	<code>page.check('#chk')</code>
<code>waitForURL()</code>	<code>page.waitForURL()</code>
<code>waitForSelector</code>	<code>page.waitForSelector()</code>
<code>evaluate()</code>	<code>page.evaluate(fn)</code>
<code>reload()</code>	<code>page.reload()</code>

Transport Mode Comparison

Transport	Best For	Port	Config Field
<code>stdio</code>	Claude Desktop, VS Code, Cursor	N/A	"type": "stdio"
<code>HTTP/SSE</code>	Cloudflare Workers, remote agents	8931	"type": "http"

Snapshot Mode

Reads accessibility tree
No screenshots needed
Fast & deterministic

Vision Mode

Uses screenshots
Needs vision-capable LLM
Slower but visual

Self-Healing Tests

AI detects broken locators
Suggests stable selectors
Reduces maintenance

Agentforce Testing

Einstein AI Agent REST API
OAuth 2.0 + session flow
Atlas Reasoning Engine

CI/CD Integration

Headless mode for pipelines
Works with GitHub Actions
npx playwright test

MCP + POM Pattern

Combine Page Objects
with MCP tool calls
Cleaner test structure

MCP vs Traditional Playwright — At a Glance

Aspect	Traditional Playwright	Playwright MCP
Test Creation	Manual scripting	Natural language prompts
Locators	Developer writes selectors	AI picks stable selectors
Adaptability	Static — breaks on change	Dynamic — adapts to changes
Learning Curve	Medium to High	Low — plain English
Best For	Stable, known flows	Exploratory & adaptive testing

Quick CLI Commands

```
npx @playwright/mcp@latest // Start MCP server (stdio)

npx @playwright/mcp@latest --port 8931 // Start MCP server (HTTP/SSE)

npx playwright test // Run Playwright test suite

npx playwright show-report // Open HTML test report
```

Found this useful? Save it | Like | Comment | Follow